

PART I

```

C:\Users\alex2\Desktop\vsCod x + v
OSTick    created, Thread ID  7748
Task[ 63] created, TCB Address 2de6a0
-----After Task[ 63] begin linked-----
Previous TCB point to address    0
Current  TCB point to address 2de6a0
Next     TCB point to address    0

The file ./TaskSet.txt was opened
Task[  1] created, TCB Address 2de6f8
-----After Task[  1] begin linked-----
Previous TCB point to address    0
Current  TCB point to address 2de6f8
Next     TCB point to address 2de6a0

Task[  2] created, TCB Address 2de750
-----After Task[  2] begin linked-----
Previous TCB point to address    0
Current  TCB point to address 2de750
Next     TCB point to address 2de6f8

=====TCB linked list=====
Task   Prev_TCB_addr  TCB_addr  Next_TCB_addr
  2         0         2de750    2de6f8
  1      2de750      2de6f8    2de6a0
 63      2de6f8      2de6a0         0

```

- 在 "Micrium / Software / uCOS-II / Ports / Win32 / Visual Studio / os_cpu_c.c" 的 OSTCBInitHook() 中修改並印出 TCB Address。

```

625     #if (OS_MSG_TRACE > 0u)
626         OS_Printf("Task[%3.1d] created, TCB Address %6x\n", p_tcb->OSTCBPrio, (void*)p_tcb);
627     #endif

```

- 在 "Micrium/Software/uCOS-II/Source/os_core.c" 的 OS_TCBInit() 印出 TCB linked 資訊

```

2147     OS_TRACE_TASK_READY(ptcb);
2148     printf("-----After Task[%3.1d] begin linked-----\n", ptcb->OSTCBPrio);
2149     printf("Previous TCB point to address %6x\n", (void*)ptcb->OSTCBPrev);
2150     printf("Current  TCB point to address %6x\n", (void*)ptcb);
2151     printf("Next     TCB point to address %6x\n\n", (void*)ptcb->OSTCBNext);
2152     OS_EXIT_CRITICAL();

```

- 在 "Microsoft/Windows/Kernel/OS2/main.c" 的 main() 印出 TCB linked list

```

137     printf("=====TCB linked list=====");
138     printf("Task \t Prev_TCB_addr  TCB_addr  Next_TCB_addr\n");
139     for (int i = OS_MAX_TASKS + OS_N_SYS_TASKS; i >= 0; i--) {
140         if (OSTCBTbl[i].OSTCBPrio != 0) {
141             printf("%2d \t %6x \t %6x \t %6x\n",
142                 OSTCBTbl[i].OSTCBPrio,
143                 OSTCBTbl[i].OSTCBPrev,
144                 &OSTCBTbl[i],
145                 OSTCBTbl[i].OSTCBNext);
146         }
147     }
148     printf("\n");

```

PART II

- 在"Micrium/Software/uCOS-II/Source/ucos_ii.h"的 task_para_set 新增變數:

TaskRemainTime:剩餘執行時間

TaskDeadLine:任務到期時間

TaskResponseTime:從到達到結束的時間

TaskDelay:應該延遲的時間

```
78  ✓      typedef struct task_para_set {
79          INT16U TaskID;
80          INT16U TaskArriveTime;
81          INT16U TaskExecutionTime;
82          INT16U TaskPeriodic;
83          INT16U TaskNumber;
84          INT16U TaskPriority;
85          INT16U TaskCount;
86          INT16U TaskRemainTime;
87          INT16U TaskDeadLine;
88          INT16U TaskResponseTime;
89          INT16U TaskDelay;
90      } task_para_set;
```

- 在"Microsoft/Windows/Kernel/OS2/app_hooks.c"的 InputFile 設定上述初始值。

```
95      while (!feof(fp)) {
96          i = 0;
97          memset(str, 0, sizeof(str));
98          fgets(str, sizeof(str) - 1, fp);
99          ptr = strtok_s(str, " ", &pTmp);
100
101          while (ptr != NULL) {
102              TaskInfo[i] = atoi(ptr);
103              ptr = strtok_s(NULL, " ", &pTmp);
104              if (i == 0) {
105                  TASK_NUMBER++;
106                  TaskParameter[j].TaskID = TASK_NUMBER;
107              }
108              else if (i == 1)
109                  TaskParameter[j].TaskArriveTime = TaskInfo[i];
110              else if (i == 2) {
111                  TaskParameter[j].TaskExecutionTime = TaskInfo[i];
112                  TaskParameter[j].TaskRemainTime = TaskInfo[i];
113              }
114              else if (i == 3) {
115                  TaskParameter[j].TaskPeriodic = TaskInfo[i];
116              }
117              TaskParameter[j].TaskDeadLine = TaskParameter[j].TaskArriveTime + TaskParameter[j].TaskPeriodic;
118              i++;
119          }
120      }
```

- 在 "Microsoft/Windows/Kernel/OS2/main.c"
- ✧ task(): 在 RM 下的執行過程，並加入 miss deadline 的處理機制。程式一開始先根據任務的抵達時間讓任務延遲啟動，之後進入主要的執行迴圈，只要系統尚未偵測到 miss deadline，就會讓任務依序執行並記錄執行情況。若任務順利執行完畢，則更新其週期性參數，包括下一次的抵達時間、回應時間等，並在任務完成時輸出執行結果。最後面的 if 用於偵測前後任務為同一個時的情況。最後的 while 用於 miss deadline 後的狀況，跳出原執行迴圈，進入無窮迴圈。

```

91 static void task(void* p_arg);
92
93 int missDeadline = 0;
94 int missTask;
95 void task(void* p_arg) {
96     task_para_set* task_data = (task_para_set*)p_arg;
97
98     int timeTag;
99     timeTag = OSTimeGet();
100
101     if (task_data->TaskArriveTime > timeTag) OSTimeOnly(task_data->TaskArriveTime - timeTag);
102
103     while (missDeadline == 0) {
104         while (task_data->TaskRemainTime > 0 && missDeadline == 0) {
105             if (missDeadline == 1) break;
106
107             LOG_print(3, "./Output.txt", "%2d task(%2d) is running!\n", OSTime, task_data->TaskID);
108             timeTag = OSTimeGet();
109
110             while (OSTimeGet() == timeTag) {
111                 if (missTask != 0 && missDeadline == 0)
112                 {
113                     missDeadline = 1;
114                     LOG_print(3, "./Output.txt", "%2d Missdeadline\ttask(%2d)(%2d)\t-----\n", OSTime, missTask, TaskParameter[missTask].TaskCount);
115                     break;
116                 }
117             }
118             if (missDeadline==1) break;
119
120
121 OS_ENTER_CRITICAL();
122 timeTag = OSTimeGet();
123
124 task_data->TaskDeadline += task_data->TaskPeriodic;
125 task_data->TaskResponseTime = timeTag - task_data->TaskArriveTime;
126 task_data->TaskArriveTime += task_data->TaskPeriodic;
127
128 if (timeTag >= task_data->TaskArriveTime) task_data->TaskDelay = 0;
129 else task_data->TaskDelay = task_data->TaskArriveTime - timeTag;
130
131 task_data->TaskRemainTime = task_data->TaskExecutionTime;
132 OS_EXIT_CRITICAL();
133
134 OSTimeOnly(task_data->TaskDelay);
135
136 if (OSTCBCur->CompletedFlag == 1 && ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay == 0) {
137     if (OSTCBHighRdy == OSTCBCur) {
138         LOG_print(3, "./Output.txt", "%2d Completion\ttask(%2d)(%2d)\ttask(%2d)\t%2d\t%2d\t\n",
139             OSTimeGet(),
140             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskID,
141             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount++,
142             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskID,
143             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskCount,
144             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime,
145             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskExecutionTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay);
146     }
147     OSTCBCur->CompletedFlag = 0;
148 }
149 }
150 }
151 while (1) {}
152

```

- ✧ main():在任務資料進入 TaskParameter 時，比較任務的 TaskPeriodic 大小，由小到大，依順序分配 TaskPriority

```
200     for (int a = 0; a < TASK_NUMBER - 1; a++) {
201         for (int b = a + 1; b < TASK_NUMBER; b++) {
202             if (TaskParameter[a].TaskPeriodic > TaskParameter[b].TaskPeriodic) {
203                 task_para_set temp = TaskParameter[a];
204                 TaskParameter[a] = TaskParameter[b];
205                 TaskParameter[b] = temp;
206             }
207         }
208     }
209
210     for (int i = 0; i < TASK_NUMBER; i++) TaskParameter[i].TaskPriority = i + 1;
```

- 在 "Micrium/Software/uCOS-II/Source/os_core.c"

- ✧ OSTimeTick:更新 TaskTimeRemain，並確認任務是否完成和到期。

```
1087     currentTime = OSTimeGet();
1088     if (ptcb == OSTCBCur && ptcb->OSTCExtPtr && ((task_para_set*)(ptcb->OSTCExtPtr))->TaskRemainTime > 0) /* Check if this is the currently running task */
1089     {
1090         (((task_para_set*)(ptcb->OSTCExtPtr))->TaskRemainTime)--; /* Decrement comp time counter for the running task */
1091         if (((task_para_set*)(ptcb->OSTCExtPtr))->TaskRemainTime == 0) {
1092             OSTCBCur->CompletedFlag = 1;
1093         }
1094     }
1095     if (ptcb->OSTCBPrio >= 1 && ptcb->OSTCBPrio <= 3 && currentTime >= ((task_para_set*)(ptcb->OSTCExtPtr))->TaskDeadline && ((task_para_set*)(ptcb->OSTCExtPtr))->TaskRemainTime > 0)
1096     {
1097         missTask = ptcb->OSTCBPrio;
1098     }
```

- ✧ OSIntExit:ISR 結束後執行的系統回復與排程流程，若發現高優先任務需搶占目前任務，則記錄 Preemption 或 Completion，同時顯示任務的回應、搶占、延遲時間，更新 TaskArrive。整體上這段程式負責中斷後的任務切換、排程決策與任務狀態紀錄，是 RM 排程核心的上下文切換控制區段。

```
712     int currentTime;
713     currentTime = OSTimeGet();
714
715     OS_ENTER_CRITICAL();
716     if (OSIntNesting > 0u) { /* Prevent OSIntNesting cur wrapping */
717         OSIntNesting--;
718     }
719     if (OSIntNesting == 0u) { /* Reschedule only if all ISRs complete ... */
720         if (OSLockNesting == 0u) { /* ... and not locked. */
721             #if ALGORITHM == RM
722                 OS_SchedNew();
723                 OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
724             #endif
725             if (OSPrioHighRdy != OSPrioCur) { /* No Ctx Sw if current task is highest rdy */
726                 #if OS_TASK_PROFILE_EN > 0u
727                     OSTCBHighRdy->OSTCBCtxSwCtr++; /* Inc. # of context switches to this task */
728                 #endif
729                 OSTCtsSwCtr++; /* Keep track of the number of ctx switches */
730                 if (OSTCBCur->CompletedFlag != 1 && missDeadline == 0) {
731                     if (OSTCBCur->OSTCBPrio != 63) {
732                         LOG_print(3, "Output.txt", "%2d Preemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t\n",
733                             OSTimeGet(),
734                             ((task_para_set*)(OSTCBCur->OSTCExtPtr))->TaskID,
735                             ((task_para_set*)(OSTCBCur->OSTCExtPtr))->TaskCount,
736                             ((task_para_set*)(OSTCBHighRdy->OSTCExtPtr))->TaskID,
737                             ((task_para_set*)(OSTCBHighRdy->OSTCExtPtr))->TaskCount
738                         );
739                     }
740                     else {
741                         LOG_print(3, "Output.txt", "%2d Preemption\ttask(%2d)\ttask(%2d)(%2d)\t\n",
742                             OSTimeGet(),
743                             63,
744                             ((task_para_set*)(OSTCBHighRdy->OSTCExtPtr))->TaskID,
745                             ((task_para_set*)(OSTCBHighRdy->OSTCExtPtr))->TaskCount
746                         );
747                     }
748                 }
749             }
```

```

749 if (OSTCBCur->CompletedFlag == 1 && missDeadline == 0) {
750     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime = currentTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskArriveTime + 1;
751     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskArriveTime += ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskPeriodic;
752
753     if (currentTime >= ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskArriveTime) ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay = 0;
754     else ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay = ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskArriveTime - currentTime;
755
756     if (OSTCBHighRdy->OSTCBPrio != 63) {
757         LOG_print(3, "./Output.txt", "%2d Completion\ttask(%2d)(%2d)\ttask(%2d)\t%ld\t%ld\t%ld\n",
758             OSTimeGet(),
759             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskID,
760             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount,
761             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskID,
762             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskCount,
763             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - 1,
764             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskExecutionTime - 1,
765             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay
766         );
767
768         ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount++;
769     }
770     else {
771         LOG_print(3, "./Output.txt", "%2d Completion\ttask(%2d)(%2d)\ttask(%2d)\t%ld\t%ld\t%ld\n", OSTimeGet(),
772             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskID,
773             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount,
774             63,
775             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - 1,
776             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskExecutionTime - 1,
777             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay
778         );
779
780         ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount++;
781     }
782     OSTCBCur->CompletedFlag = 0;
783     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskArriveTime += ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskPeriodic;
784 }

```

✧ OS_Sched:當任務呼叫 OSTimeDly 時，任務會讓出 CPU，此時如果任務完成便會印出訊息如前述之。

```

1813 if (OSTCBCur->CompletedFlag == 1 && missDeadline == 0) {
1814     if (OSTCBHighRdy->OSTCBPrio != 63) {
1815         LOG_print(3, "./Output.txt", "%2d Completion\ttask(%2d)(%2d)\ttask(%2d)\t%ld\t%ld\t%ld\n",
1816             OSTimeGet(),
1817             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskID,
1818             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount,
1819             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskID,
1820             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))>TaskCount,
1821             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime,
1822             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskExecutionTime,
1823             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay
1824         );
1825
1826         ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount++;
1827     }
1828     else {
1829         LOG_print(3, "./Output.txt", "%2d Completion\ttask(%2d)(%2d)\ttask(%2d)\t%ld\t%ld\t%ld\n",
1830             OSTimeGet(),
1831             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskID,
1832             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount,
1833             63,
1834             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime,
1835             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskResponseTime - ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskExecutionTime,
1836             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskDelay
1837         );
1838
1839         ((task_para_set*)(OSTCBCur->OSTCBExtPtr))>TaskCount++;
1840     }
1841     OSTCBCur->CompletedFlag = 0;
1842 }

```

PART III

➤ 未完成